

AgentCore Gateway

What is Gateway

Goal

Gateway targets (supported targets):

Why we choose MCP Server target type

Connecting an MCP server as a lambda target

What is Lambda Web Adapter:

steps to setup lambda web adapter to MCP server hosted in lambda

Why not mangum

Tool Discovery and Semantic Search

Tool Discovery (MCP targets only)

Semantic Search (Gateway-wide)

Gateway Authorisation: Two Independent Concepts

Inbound authorisation

Outbound authorization

Resolved context propagation from Runtime to target or day-2-lambda

Why the Gateway uses AWS IAM instead of Custom JWT

Hops in the Request pipeline

Resources involved

Why we need to go with Web Adapter Lambda instead of Mangum:

Mangum setup

Mangum command and response

Lambda web adapter setup

web adapter command and response

Learnings through failed experiments:

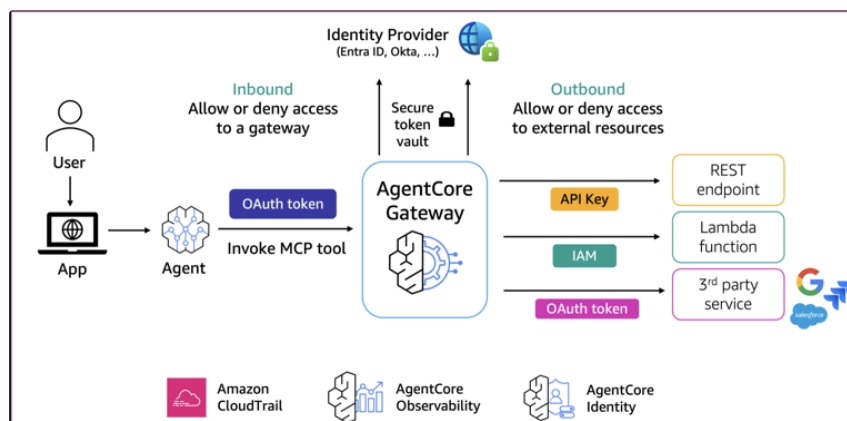
Connection to day-2-MCP using AgentCore Gateway with static API-key headers (MCP-server type):

What is Gateway

The Gateway is a fully managed service that acts as a secure, intelligent router between agents and the tools they need to call. Instead of agents calling external services directly each with different authentication methods and formats agents make a single connection to the Gateway, which handles everything else.

Goal

This exploration aims to securely connect day-2-MCP lambda for each user and propagate user context from runtime to day-2-MCP lambda.



Gateway targets ([supported targets](#)):

Targets define the tools that your AgentCore gateway will host. AgentCore gateway supports two categories of targets:

- **MCP targets:** operate in aggregation mode. AgentCore gateway acts as an MCP server whose capabilities combine those of all its MCP targets into a single unified virtual MCP server. (Tried only this specific our use case)
- **HTTP targets:** AgentCore gateway sends traffic directly to HTTP targets without aggregation or protocol translation. You define tools using an OpenAPI or Smithy schema so that agents can discover and invoke them.

We can connect to these target types as shown below, but we have explored only specific requirement i.e connecting to a lambda which has multiple tools.

The screenshot shows the 'Add Target' form with the following fields and options:

- Select a target protocol:** Radio buttons for 'MCP target' (selected) and 'Agent target - new'. The MCP target description says: 'The MCP target can be defined as an MCP server, API gateway, Lambda, or REST API to define MCP tools.' The Agent target description says: 'The HTTP target acts as a fronting layer to an AgentCore runtime through the HTTP protocol.'
- Target name:** Text input with 'target-quick-start-mo9gei' entered. A note below says: 'Valid characters are a-z, A-Z, 0-9, and - (hyphen). The name can have up to 50 characters.'
- Target description - optional:** Text area with 'Enter description' placeholder.
- Target type:** Radio buttons for 'MCP server' (selected), 'Lambda ARN', 'REST API', 'API Gateway', and 'Integrations'.
 - 'MCP server': Define this target with pre-configured MCP server.
 - 'Lambda ARN': Define this target with a Lambda function and S3 resource.
 - 'REST API': Define this target with an OpenAPI or Smithy schema.
 - 'API Gateway': Define this target with an API Gateway REST API deployed to a stage.
 - 'Integrations': Define this target with pre-configured templates from integration providers.
- MCP endpoint:** Text input with 'Enter endpoint' placeholder. A note says: 'The location of the MCP server associating one or more tools to this target.'
- Enable MCP resource priority - new:** Checkbox (unchecked). A note says: 'When enabled, you can specify a priority for this target as defined in the MCP protocol.'
- MCP listing mode - new:** Radio buttons for 'Default' (selected) and 'Dynamic'.
 - 'Default': Select how this target will list resources. A link 'Learn more i2' is present. Description: 'MCP tools in this target are cached at the control plane for faster access. This MCP target will need to be synced.'
 - 'Dynamic': Description: 'MCP resources in this target will be dynamically retrieved when listing tools. This MCP target will not need to be synced. Dynamically listed resources in this target cannot be searched.'

Why we choose MCP Server target type

We configured the target with MCP as the protocol and selected Lambda ARN as the target type, since our day-2 MCP server is hosted in Lambda. This approach comes with two caveats:

- Every tool exposed by the day-2 MCP Lambda must be defined explicitly.
- We lose MCP listing mode, so the available tools can't be enumerated dynamically at runtime.

Connecting an MCP server as a lambda target

To replicate day-2 MCP, we have used similar MCP setup in the lambda with some of tools like currency converter, tip calculator as tools. For MCP setup have used FastMCP, UV corn packages and also [lambda web adapter](#).

What is Lambda Web Adapter:

AWS Lambda Web Adapter enables running standard web applications on AWS Lambda. It bridges Lambda's event-driven model and your HTTP-based web app. Normally, lambda requires handling lambda-specific event formats. Web Adapter removes this constraint and also supports response streaming.

steps to setup lambda web adapter to MCP server hosted in lambda

▼ steps to setup lambda web adapter

```
1 Write your mcp_server code in mcp_server.py
2
3 rm -rf mcp-deploy mcp-deploy.zip
4 mkdir mcp-deploy && cd mcp-deploy
5
6 # Force Linux x86_64 platform
7 pip install fastmcp uvicorn \
8   --platform manylinux2014_x86_64 \
```

```

9      --only-binary=:all: \
10     --target . \
11     --python-version 3.12
12
13 # Copy your mcp_server.py here
14 cp /path/to/mcp_server.py .
15
16 # Create run.sh
17 cat > run.sh << 'EOF'
18 #!/bin/bash
19 exec python mcp_server.py
20 EOF
21 chmod +x run.sh
22
23 # Zip
24 zip -r ../mcp-deploy.zip .
25 cd ..
26
27 # Upload
28 aws lambda update-function-code \
29     --function-name weather-agent-lambda-dev-weatherAgent \
30     --zip-file fileb://mcp-deploy.zip \
31     --region us-east-1
32
33 aws lambda update-function-configuration \
34     --function-name weather-agent-lambda-dev-weatherAgent \
35     --layers \
36     "arn:aws:lambda:us-east-1:041875151542:layer:weather-agent-dependencies:9" \
37     "arn:aws:lambda:us-east-1:753240598075:layer:LambdaAdapterLayerX86:27" \
38     --handler run.sh \
39     --timeout 30 \
40     --memory-size 512 \
41     --environment "Variables={PORT=8080,AWS_LAMBDA_EXEC_WRAPPER=/opt/bootstrap,AWS_LWA_INVOKE_MODE=response_stream}" \
42     --region us-east-1

```

Why not mangum

Since we are familiar with the Mangum adapter, we first tried setting it up by removing Lambda Web Adapter and ran into this error.

- ✖ error while connecting gateway with mangum hosted mcp server

There was an error creating the target(s). Error: Gateway target created with status: FAILED - Failed to connect and fetch tools from the provided MCP target server. Error - Invalid SSE response. Status code: 200 Line: {"statusCode": 200, "body": "event: message\r\nndata: {"jsonrpc":"2.0","id":"07a5f433-0","result":{"protocolVersion":"2025-06-18","capabilities":{"experimental":{},"logging":{},"prompts":{"listChanged":true},"resources":{"subscribe":false,"listChanged":true},"tools":{"listChanged":true},"extensions":{"io.modelcontextprotocol/ui":{}}},"serverInfo":{"name":"Utility Tools MCP Server","version":"3.3.0"}}}\r\n\r\n", "headers": {"cache-control": "no-cache, no-transform", "connection": "keep-alive", "content-type": "text/event-stream", "x-accel-buffering": "no"}, "isBase64Encoded": false}

Mangum is an ASGI-to-Lambda adapter. It receives the Lambda event, converts it into ASGI `scope`, `receive`, and `send` calls, passes them to your FastAPI/FastMCP ASGI app, collects the response, and returns a Lambda proxy response (`statusCode`, `headers`, `body`, etc.). But MCP uses Server-Sent Events (SSE) with `content-type: text/event-stream`. AgentCore Gateway requires `RESPONSE_STREAM` mode to handle SSE properly, because SSE is inherently a streaming protocol the server sends events incrementally. With `RESPONSE_STREAM`, the Function URL doesn't unwrap anything. It passes the raw Lambda output directly to the caller. So what the caller (AgentCore

Gateway) receives is the Mangum JSON envelope as a literal string. Gateway tries to parse this as SSE, sees it starts with "statusCode" instead of event: , and fails with "Invalid SSE response as above.

Tool Discovery and Semantic Search

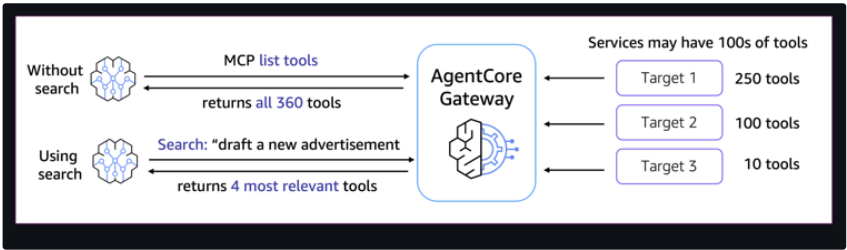
Tool Discovery (MCP targets only)

When an MCP server is added as a Gateway target, the Gateway connects to it and downloads all tool definitions, this is called implicit synchronization and happens automatically during target creation. If the MCP server later adds, modifies, or removes tools, the Gateway does not automatically know. A manual explicit synchronization must be triggered via the SynchronizeGatewayTargets API to refresh the tool catalog.

The listing mode for MCP targets can be set to either DEFAULT or DYNAMIC. In DYNAMIC mode, clients discover MCP server capabilities when a user invokes an MCP operation the Gateway retrieves capabilities live by forwarding requests to the MCP server. In DEFAULT mode, the Gateway serves from its pre-fetched cache. Note that DYNAMIC mode is incompatible with semantic search.

Semantic Search (Gateway-wide)

Semantic search is a capability at the Gateway level, not specific to any one target type. As organizations scale to hundreds or thousands of tools, discovering the right tool becomes increasingly challenging. When agents are presented with too many tools simultaneously, they can experience "tool overload", leading to hallucinations, incorrect tool selections, or inefficient execution paths. When semantic search is enabled, the Gateway pre-computes vector embeddings for all tool definitions across all targets and exposes a built-in tool called x_amz_bedrock_agentcore_search . Agents can call this tool with a natural language query to get back only the most relevant tools for the task at hand, rather than receiving the full tool list every time.



Gateway Authorisation: Two Independent Concepts

The Gateway has two completely separate authorization settings that are often confused:

Inbound authorisation

Controls who is allowed to call the Gateway. This protects the Gateway endpoint from unauthorized access.

Option	How It Works	Best For	experiment ed in POC
None	No authentication required Gateway is publicly accessible	Development and testing	Yes
AWS IAM	Callers must sign requests using	Agents running on AWS	Yes

	AWS IAM credentials		
Custom JWT	Callers must provide a valid JWT token from an OIDC provider (Cognito, Auth0, Okta)	External apps and non-AWS callers	yes

Outbound authorization

Controls how the Gateway authenticates when calling a target tool or service. This is completely independent of inbound auth.

Option	How It Works	Best For	experimented in POC
None	No credentials sent target must be publicly accessible	Development, or when credentials are passed as headers	yes
AWS IAM	Gateway signs requests with its IAM role	AWS services and Lambda	yes
API Key	Gateway reads a key from Secrets Manager and injects it into requests	APIs protected by API keys	No
OAuth 2LO	Gateway fetches a machine-to-machine token from an OAuth server	Enterprise APIs with OAuth	No
OAuth 3LO	Gateway exchanges a user's identity token for a service-specific token	Per-user access control	No

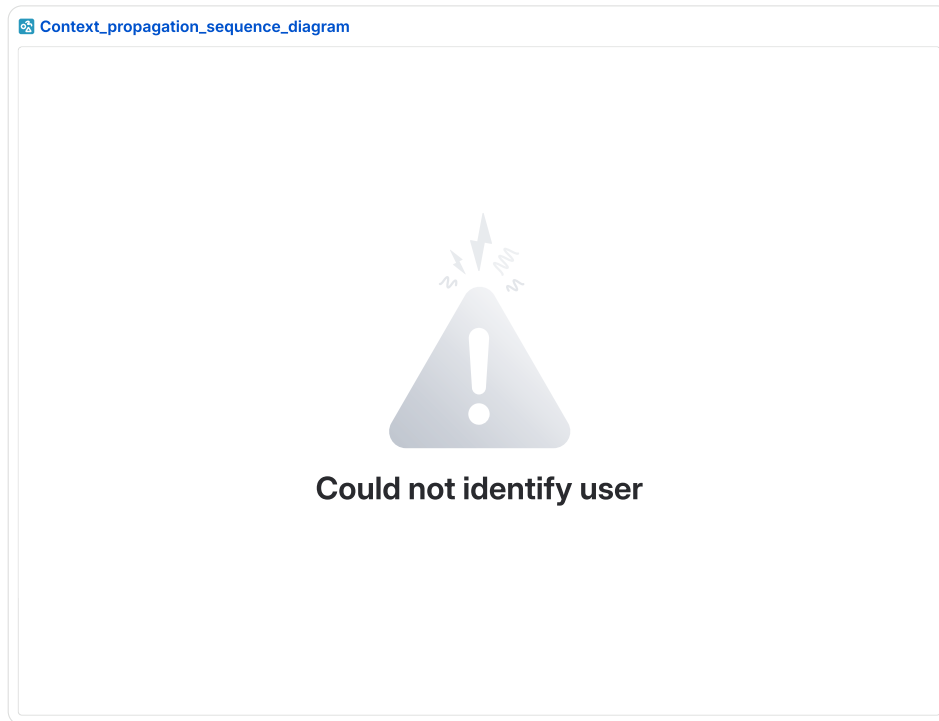
Resolved context propagation from Runtime to target or day-2-lambda

The goal here is the propagate user context from AgentCore Runtime to AgentCore gateway and then to the target lambdas.

Why the Gateway uses AWS IAM instead of Custom JWT

AWS IAM locks the Gateway to only the Runtime's specific IAM role. No Cognito token, no matter how valid, can call this Gateway directly. The Runtime becomes the only trusted entry point.

Below is the sequence flow of user context from user to target lambda.



- The gateway uses AWS_IAM auth, So all the requests now hitting the gateway should be SigV4 signed, else it returns 403 error. The runtimes IAM role is what proves that it is a trusted runtime to the gateway and botocore handles the signing automatically using the containers temporary instance credentials.
- Three separate grants are needed for the pipeline to work. A common misconception is that one side of the permission is enough both sides must always exist.

When your Runtime calls the Gateway URL via httpx, it is making a plain HTTP request. AWS has no way of knowing who is making that request it is just an anonymous HTTP call hitting a URL. SigV4 is the mechanism that proves to the Gateway that this request actually came from a specific AWS role. It attaches your AWS identity to the HTTP request itself, which is why it is required even though the Gateway is already configured with AWS IAM auth.

▼ IAM permissions required

Runtime Role → Gateway, without this the runtime's SigV4 signed requests get 403 even though the gateway is AWS_IAM

```
1 aws iam put-role-policy \
2   --role-name <Runtime-role-arn> \
3   --policy-name AllowIamGatewayInvoke \
4   --policy-document '{
5     "Version": "2012-10-17",
6     "Statement": [{
7       "Effect": "Allow",
8       "Action": "bedrock-agentcore:InvokeGateway",
9       "Resource": "<gateway_resource_arn>"
10    }]
11  }'
```

Lambda resource based policy- AWS requires that when a service (`bedrock-agentcore`) invokes a Lambda, the Lambda must explicitly allow it. Without this, the gateway can't invoke the interceptor and returns 500:

```
1 aws lambda add-permission \
2   --function-name Interceptor-test \
3   --statement-id AllowIamGatewayInvoke \
4   --action lambda:InvokeFunction \
5   --principal bedrock-agentcore.amazonaws.com \
6   --source-arn <gateway_resource_arn>
```

Gateway execution role → Interceptor - the gateway doesn't invoke the interceptor using your credentials, it uses its own execution role. That role has no permission to call your Lambda by default. Both sides of the permission must exist resource-based policy alone is not enough:

```
1 aws iam put-role-policy \
2   --role-name <Gateway_execution_role> \
3   --policy-name AllowInterceptorInvoke \
4   --policy-document '{
5     "Version": "2012-10-17",
6     "Statement": [{
7       "Effect": "Allow",
8       "Action": "lambda:InvokeFunction",
9       "Resource": "<Interceptor_lambda_arn>"
10    }]
11  }'
```

▼ Attaching the Interceptor via SDK

The AWS CLI does not support all AgentCore Gateway operations. Use `botocore` to attach the interceptor. Note that `passRequestHeaders: True` is required without it, the `X-Resolved-Context` header is not forwarded to the Interceptor.

- Had to use the SDK approach and add the configuration

```

1 client.update_gateway(
2     gatewayIdentifier="<gateway_name>",
3     name=gateway["name"],
4     roleArn=gateway["roleArn"],
5     protocolType=gateway["protocolType"],
6     authorizerType=gateway["authorizerType"],
7     interceptorConfigurations=[{
8         "interceptor": {
9             "lambda": {"arn": "<interceptor_lambda_arn>"}
10        },
11        "interceptionPoints": ["REQUEST"],
12        "inputConfiguration": {"passRequestHeaders": True}
13    }]
14 )

```

- An example overview of what interceptor receives and what it responds with

- Receives

```

1 {
2     "mcp": {
3         "gatewayRequest": {
4             "headers": {
5                 "X-Resolved-Context": "{\"sub\": \"04b8c4a8...\", \"username\": \"testuser\"}"
6             },
7             "body": { "method": "tools/call", "params": {...} }
8         }
9     }
10 }

```

- Responds with

```

1 {
2     "interceptorOutputVersion": "1.0",
3     "mcp": {
4         "transformedGatewayRequest": {
5             "body": {
6                 "method": "tools/call",
7                 "params": {
8                     "name": "currency_converter",
9                     "arguments": {
10                        "amount": 100,
11                        "from_currency": "USD",
12                        "to_currency": "INR",
13                        "_caller": {"sub": "04b8c4a8...", "username": "testuser", ...}
14                    }
15                }
16            }
17        }
18    }
19 }

```

```

1 Runtime (execution role)
2 → resolves user identity from JWT claims ONCE → stored as resolved_context dict
3 → sets X-Resolved-Context: {"sub":..., "username":...} header
4 → signs request with SigV4 using its own credentials
5 → Gateway receives request
6 → validates SigV4 signature ✓
7 → checks RuntimeRole has bedrock-agentcore:InvokeGateway ✓
8 → calls Interceptor Lambda (REQUEST interception point)
9 → Interceptor reads X-Resolved-Context header
10 → injects _caller into tools/call arguments
11 → returns transformedGatewayRequest
12 → Gateway forwards modified request to helloWorld Lambda
13 → Lambda pops _caller → user_context
14 → routes to tool handler with user identityG

```

SigV4 is required

How the Interceptor Lambda works

The Interceptor is a lightweight Lambda function that sits between the Gateway and the target Lambda. Every request passing through the Gateway hits the Interceptor first at the REQUEST interception point before being forwarded to the tool.

Hops in the Request pipeline

```
1 User
2   ↓ (cognito login, jwt auth)
3 AgentCore Runtime
4   ↓(IAM role, SigV4 sign)
5 AgentCore Gateway
6   ↓
7 Interceptor Lambda
8   ↓
9 helloWorld Lambda(mcp example)
```

Resources involved

1. Cognito user pool
2. Application client
3. Runtime
4. Gateway
5. Interceptor lambda (additional lambda to propagate requests)
6. Target lambda (MCP server)

Why we need to go with Web Adapter Lambda instead of Mangum:

Mangum setup

▼ mangum code setup

```
1 # Create ASGI app and wrap with Mangum
2 # -----
3 # Get the FastMCP ASGI app
4 mcp_app = mcp.http_app(path="/mcp", stateless_http=True)
5
6 # Wrap in a Starlette app, passing the FastMCP lifespan
7 # This is critical without lifespan, FastMCP's task group never initializes
8 asgi_app = Starlette(
9     routes=mcp_app.routes,
10    lifespan=mcp_app.lifespan,
11 )
12
13 # Mangum translates Lambda Function URL events <-> ASGI
14 handler = Mangum(asgi_app, lifespan="on")
15
16
```

Mangum command and response

▼ Mangum command

```
1 Magnum
2 curl -X POST "https://u34tzm7pkjvi524bnk3g4xdfia0hxmtr.lambda-url.us-east-1.on.aws/mcp" \
3     -H "Content-Type: application/json" \
```

```
4 -H "Accept: application/json, text/event-stream" \  
5 -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}' \  
6
```

- ✓ Mangum response

```
1 [{"statusCode": 200, "body": {"event": "message\r\nndata: {\r\n  \"jsonrpc\": \"2.0\", \"id\": 1, \"result\": {\r\n    \"tools\": [\r\n      {\r\n        \"name\": \"currency_converter\", \"description\": \"Convert an amount from one currency to another.\r\n\r\nSupported currencies: USD, INR, EUR, GBP, JPY.\", \"inputSchema\": {\r\n          \"additionalProperties\": false, \"properties\": {\r\n            \"amount\": {\r\n              \"default\": 1.0, \"type\": \"number\", \"description\": \"The amount to convert. Defaults to 1.0.\", \"from_currency\": {\r\n                \"default\": \"USD\", \"type\": \"string\", \"description\": \"Source currency code (e.g. USD, EUR). Defaults to USD.\", \"to_currency\": {\r\n                  \"default\": \"INR\", \"type\": \"string\", \"description\": \"Target currency code (e.g. INR, GBP). Defaults to INR.\", \"type\": \"object\", \"outputSchema\": {\r\n                    \"additionalProperties\": true, \"type\": \"object\", \"meta\": {\r\n                      \"fastmcp\": {\r\n                        \"tags\": []\r\n                      }\r\n                    }\r\n                  }, \"name\": \"unit_converter\", \"description\": \"Convert a value between units.\r\n\r\nSupported: km<->miles, kg<->lbs, meters<->feet, celsius<->fahrenheit.\", \"inputSchema\": {\r\n                    \"additionalProperties\": false, \"properties\": {\r\n                      \"value\": {\r\n                        \"type\": \"number\", \"description\": \"The numeric value to convert.\", \"from_unit\": {\r\n                          \"type\": \"string\", \"description\": \"Source unit (km, miles, kg, lbs, meters, feet, celsius, fahrenheit).\", \"to_unit\": {\r\n                            \"type\": \"string\", \"description\": \"Target unit.\", \"required\": [\r\n                              \"value\", \"from_unit\", \"to_unit\", \"type\": \"object\", \"outputSchema\": {\r\n                                \"additionalProperties\": true, \"type\": \"object\", \"meta\": {\r\n                                  \"fastmcp\": {\r\n                                    \"tags\": []\r\n                                  }\r\n                                }, \"name\": \"tip_calculator\", \"description\": \"Calculate tip and split the total bill among people.\", \"inputSchema\": {\r\n                                  \"additionalProperties\": false, \"properties\": {\r\n                                    \"bill_amount\": {\r\n                                      \"type\": \"number\", \"description\": \"The total bill amount before tip.\", \"tip_percentage\": {\r\n                                        \"default\": 15.0, \"type\": \"number\", \"description\": \"Tip percentage (e.g. 15 for 15%). Defaults to 15.\", \"num_people\": {\r\n                                          \"default\": 1, \"type\": \"integer\", \"description\": \"Number of people splitting the bill. Defaults to 1.\", \"required\": [\r\n                                            \"bill_amount\", \"type\": \"object\", \"outputSchema\": {\r\n                                              \"additionalProperties\": true, \"type\": \"object\", \"meta\": {\r\n                                                \"fastmcp\": {\r\n                                                  \"tags\": []\r\n                                                }\r\n                                              }\r\n                                            }\r\n                                          }, \"headers\": {\r\n                                            \"cache-control\": \"no-cache, no-transform\", \"connection\": \"keep-alive\", \"content-type\": \"text/event-stream\", \"x-accel-buffering\": \"no\", \"isBase64Encoded\": false\r\n                                          }\r\n                                        }\r\n                                      }\r\n                                    }\r\n                                  }\r\n                                }\r\n                              }\r\n                            }\r\n                          }\r\n                        }\r\n                      }\r\n                    }\r\n                  }\r\n                }\r\n              }\r\n            }\r\n          }\r\n        }\r\n      }\r\n    ]\r\n  }\r\n}}\r\n\r\n\"}]
```

- ✓ Mangum response error:

```
1 There was an error creating the target(s).
2 Invalid SSE response. Status code: 200 Line: {"statusCode": 200, "body": "event: message..."}
```

Gateway is receiving the **raw Lambda proxy envelope** as if it were the SSE stream. It sees `{"statusCode": 200, "body": "event: message..."}` as a single line of text which is not valid SSE format . The actual SSE content is inside the body field as a string.

Lambda web adapter setup

- ✓ lambda web adapter code-snippet for setup has solved the issue with same http_app method:

```

1 # -----
2 # Entry point uvicorn serves the ASGI app
3 # Lambda Web Adapter forwards requests here
4 # -----
5 if __name__ == "__main__":
6     host = os.environ.get("HOST", "0.0.0.0")
7     port = int(os.environ.get("PORT", "8080"))
8     print(f"Starting MCP server on http://{host}:{port}/mcp")
9     app = mcp.http_app(stateless_http=True)
10    uvicorn.run(app, host=host, port=port)

```

web adapter command and response

- ▼ command

```
1 ~ curl -X POST "https://u34tzm7pkjvi524bnk3g4xdfia0hrmtr.lambda-url.us-east-1.on.aws/mcp/" \
2 -H "Content-Type: application/json" \
3 -H "Accept: application/json, text/event-stream" \
4 -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}'
```

▼ response

```
1 data: {"jsonrpc": "2.0", "id": 1, "result": {"tools": [{"name": "currency_converter", "description": "Convert an amount from one currency to another.\n\nSupported currencies: USD, INR, EUR, GBP, JPY.", "inputSchema": {"additionalProperties": false, "properties": {"amount": {"default": 1.0, "type": "number", "description": "The amount to convert. Defaults to 1.0."}, "from_currency": {"default": "USD", "type": "string", "description": "Source currency code (e.g. USD, EUR). Defaults to USD."}, "to_currency": {"default": "INR", "type": "string", "description": "Target currency code (e.g. INR, GBP). Defaults to INR."}, "type": "object"}], "outputSchema":
```

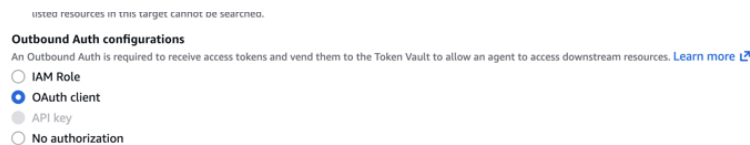
```
{
  "additionalProperties": true,
  "type": "object",
  "_meta": {
    "fastmcp": {
      "tags": []
    }
  },
  "name": "unit_converter",
  "description": "Convert a value between units.\n\nSupported: km<->miles, kg<->lbs, meters<->feet, celsius<->fahrenheit.",
  "inputSchema": {
    "additionalProperties": false,
    "properties": {
      "value": {
        "type": "number",
        "description": "The numeric value to convert.",
        "from_unit": {
          "type": "string",
          "description": "Source unit (km, miles, kg, lbs, meters, feet, celsius, fahrenheit).",
          "to_unit": {
            "type": "string",
            "description": "Target unit."
          },
          "required": [
            "value",
            "from_unit",
            "to_unit"
          ],
          "type": "object",
          "outputSchema": {
            "additionalProperties": true,
            "type": "object",
            "_meta": {
              "fastmcp": {
                "tags": []
              }
            },
            "name": "tip_calculator",
            "description": "Calculate tip and split the total bill among people.",
            "inputSchema": {
              "additionalProperties": false,
              "properties": {
                "bill_amount": {
                  "type": "number",
                  "description": "The total bill amount before tip.",
                  "tip_percentage": {
                    "default": 15.0,
                    "type": "number",
                    "description": "Tip percentage (e.g. 15 for 15%). Defaults to 15."
                  },
                  "num_people": {
                    "default": 1,
                    "type": "integer",
                    "description": "Number of people splitting the bill. Defaults to 1."
                  },
                  "required": [
                    "bill_amount"
                  ],
                  "type": "object",
                  "outputSchema": {
                    "additionalProperties": true,
                    "type": "object",
                    "_meta": {
                      "fastmcp": {
                        "tags": []
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Learnings through failed experiments:

▼ Thing need to know before setting MCP

Connection to day-2-MCP using AgentCore Gateway with static API-key headers (MCP-server type):

Explored connecting day-2-agent, Since we can connect our day-2 agent MCP server using ([MCP Server Configuration Guide - for MontyCloud team](#)) API secret key and access key it's not possible to connect to I am and O-auth since we have API.



Ran into this error after trying to connect Later learnt that

That JSON snippet is a MCP-side config that spawns `mcp-remote` to talk to the actual server at `https://api.montycloud.com/mcp` with static API-key headers. The Gateway doesn't speak that format and can't reach a stdio process it only registers HTTP MCP endpoints and its outbound auth choices for an MCP-server target are:

- NoAuth
- OAuth2 (two-legged client_credentials, or three-legged auth code)
- IAM / SigV4

Failure reason

Resource handler returned message: "GatewayTarget T8G3R2CWRR failed to stabilize, status: FAILED, reason: Failed to connect and fetch tools from the provided MCP target server. Error - {"message":"Unauthorized"}" (RequestToken: 2be8cb55-1b33-4aa2-0d0c-ef5f79367fd4, HandlerErrorCode: NotStabilized)

So we have two options:

- 1 option. add OAuth2 or SigV4 to the MontyCloud MCP server itself, so we can connect directly.
- 2 option. put a small proxy in front whose only job is to terminate the auth model the Gateway speaks (OAuth2 or IAM) and forward to MontyCloud with the static API-key headers injected from Secrets Manager.

Tried second option, got this error message

The Lambda forwards correctly but Day2's API Gateway has a resource policy that denies the Lambda's source IP. That's an AWS API Gateway message

```
{
  "Message": "User is not authorized to access this resource with an explicit deny in an identity-based policy"
}
```